

1. Part1 DSP-HW1 Introduction Peripherals

1.1 Peripherals for DSP Applications

- Analog Signal:** Continuous-time, continuous-amplitude signal.
- Digital Signal:** Discrete-time, discrete-amplitude signal.
- ADC Process:**
 - Sampling: converts continuous-time signal to discrete-time signal.
 - Quantization: converts continuous-amplitude signal to discrete-amplitude signal (finite precision).
- Sample and Hold (S/H):**
 - output updates discretely at times 1,2,3,..., equal to the instantaneous value of input

1.2 Sampling

Uniform sampling / Periodic sampling

- sampling interval T ; sampling frequency $f_s = 1/T$.
- Input sinusoid: $x_c(t) = \sin(2\pi f t)$
- Discrete-time output signal $x(n) = \sin(2\pi(f_0 + kf_s)nT) = \sin(2\pi n(f_0/f_s + k))$
 - Frequencies f_0 and $f_0 + kf_s$ (k integer) become indistinguishable after sampling.

Sampling low-pass signals

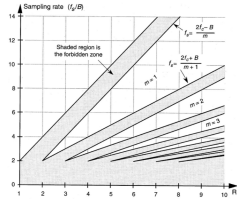
- Nyquist Sampling Theorem: a bandlimited low-pass signal $x_a(t)$ with bandwidth B can be perfectly reconstructed if $f_s > 2B$. The frequency $f_s/2$ is the **Nyquist frequency**.
- Aliasing: occurs when $f_s \leq 2B$.

Sampling band-pass signals

- Applied to signals centered at frequency f_c with bandwidth B ($f_c \gg B$).
- Non-aliasing condition for integer $m \geq 0$:

$$\frac{2f_c + B}{m+1} \leq f_s \leq \frac{2f_c - B}{m}, \quad f_s \geq 2B.$$

- Regions of possible band-pass sampling rates



1.3 Peripherals

Anti-Aliasing Filter:

- Placed **before** ADC.
- Purpose: limit analog signal bandwidth to $\leq f_s/2$ to prevent aliasing.
- Must be used even for bandlimited signals to remove out-of-band noise.
- For band-pass sampling: use a **band-pass** filter centered at f_c with bandwidth B .

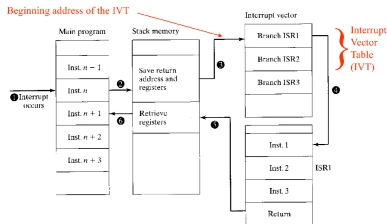
Reconstruction Filter (Anti-Imaging Filter):

- Placed **after** DAC.
- Purpose: smooth staircase output (Zero-Order Hold) and remove spectral images.
- Analog low-pass filter with cutoff frequency B .
- In practice, $f_s > 2B$ is often chosen to ease filter design.

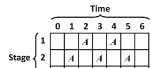
DSP processor on-chip peripherals

- Host Port Interface (HPI): enables data exchange between host processor (e.g., MCU) and DSP via shared on-chip memory.
- Direct Memory Access (DMA) Controller: manages data transfers between memory and peripherals without CPU intervention.
- Serial Port: slow data transfer (e.g., 16-bit); clock derived from data stream.
- Parallel Port: faster data transfer (e.g., 8/16-bit); requires more pins and handshake signals; often multiplexed with main data bus.
- Hardware Timer: programmable timer for generating periodic interrupts or software-controlled signal generation.
- Clock Generator: provides master clock; sourced externally or from external crystal + internal oscillator.
- Phase-Locked Loop (PLL): generates high-frequency internal clock from low-frequency external clock.
 - Advantage: reduces electromagnetic interference (EMI), allows use of cheaper external clock circuits.

1.4 Real-Time Operations: Interrupts



1.5 Real-Time Operations: Scheduling



- Permissible initiation: An initiation that does not cause collision.
- Forbidden initiation: An initiation that causes collision.
- Initiation/Permissible sequence (distance between successive initiations): 1,7,1,7,...
- Forbidden set (time) = {0,2,4,6}
- Average initiation (distance within a cycle of the initiation sequence / number of initiations) = (1+7)/2 = 4 clocks/sample
- Average throughput = 1/Average initiation
- Latency = 7 clocks, Number of utilized stage 1 = 2
- stage utilization = Number of utilized stage / Latency = 2/7
- Maximum achievable (average) throughput $\leq 1/\text{maximum number of utilized stages in a single row} = 1/3$.

2. Advanced Arithmetic Hardware

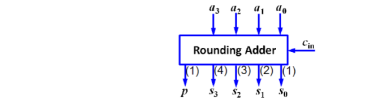
2.1 Adder

Rounding Adder, Half Adder (s.c: 1 gate delay)

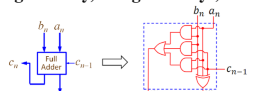


- Delay: 1 gate delay.

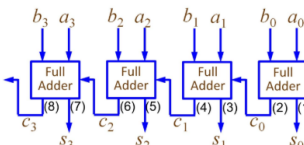
Carry Look-Ahead in Rounding Adder



$p = a_3 \cdot a_2 \cdot a_1 \cdot a_0, c_{out} = p \cdot c_{in}$
Full Adder (s: 1 gate delay, c: 2 gate delays)

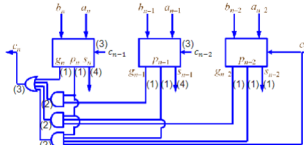


$s = a \oplus b \oplus c_{n-1}, c_n = (a \cdot b) + (a \cdot c_n) + (b \cdot c_n)$
Ripple Carry Adder (RCA)



- Structure: N full adders cascaded.
- Operation: carry propagates sequentially from LSB to MSB.
- Delay: computing c_{N-1} requires $2N$ gate delays.
- Note: delay scales linearly with bit width ($O(N)$).

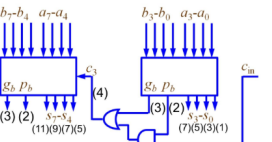
Bit-wise Carry Look-Ahead (CLA)



$$g_i = a_i b_i, \quad p_i = a_i \oplus b_i$$

- Core idea: compute carries in parallel using generate (g_i) and propagate (p_i) signals.
- Delay:
 - compute g_i, p_i in 1 gate delay;
 - compute c_i in 3 gate delays (AND-OR logic);
- Note: delay is constant $O(1)$, independent of bit width.

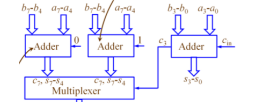
Block Carry Look-Ahead Adder



$$g_b = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0, p_b = p_3 p_2 p_1 p_0$$
$$c_{out} = G_b + P_b c_{in}$$

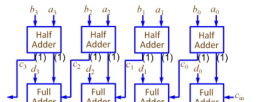
- Structure: groups bits into blocks (e.g., 4-bit); applies CLA within and between blocks.

Carry Select Adder



- Core idea: precompute results for $c_{in} = 0$ and $c_{in} = 1$, then select with MUX.
- c_7 delay = $c_3 + 1$ MUX delay; c_{11} delay = $c_3 + 2$ MUX delays.
- Note: Delay of c_3 can be reduced using CLA.

Carry Save Adder (CSA)



- Function: reduces three numbers to two numbers (sum and carry vectors) in one step.
- Only the carry bits for the last row ripple "horizontally".

Pipelined Adder



- Adding 100 32-bit data: pipeline requires $32 + (99-1)$ clocks, horizontal adder requires 32×99 clocks.
- Advantage: high throughput (new input every clock cycle).
- Disadvantage: increased latency and area (registers).

Floating-Point Adder

- Compare Exponent of A,B $\rightarrow$ Data Alignment $\rightarrow$ Add Mantissa $\rightarrow$ Determin number of leading zeros $\rightarrow$ Normalization

2.2 Multiplier

Booth Multiplier

- Precompute:
$$A_+ = A, n+1 \text{ zeros}; A_- = -A, n+1 \text{ zeros};$$
$$P = n \text{ zeros}, B, 1 \text{ zero}$$

- Check 2 LSBs of P :
 - If LSBs = 01, then $P \leftarrow P + A_+$
 - If LSBs = 10, then $P \leftarrow P + A_-$
 - If LSBs = 00 or 11, do nothing
- Shift P right by 1 bit with sign extension
- loop step 2&3 for n times
- Final product = P without the implicit bit (first $2n$ bits)

Wallace Tree Multiplier



- m_i is the number of partial products at the i -th position
- After a reduction layer, m_i reduces to $\lceil \frac{m_i}{3} \rceil + \text{round}(\frac{m_i-1}{3})$

3. Algorithms and Architectures for VLSI

3.1 Algorithm Strength Reduction

Strength Reduction

- Strong(Weak) operations: computationally (in)expensive operations.
- Strength reduction: replace strong operations with weak operations to increase throughput, reduce area, or reduce power consumption.

Fast Fourier Transform

- DFT: complex multiply: N^2 , complex add: $N(N-1)$, $X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}$,
- Polyphase Parallel FIR Filters: divide-and-conquer algorithm (Decimation-in-Time).
 - $X(k) = X_0(k) + W_N^k X_1(k)$ where $W_N^k = e^{-j2\pi k/N}$, $X_0(z) = \sum x(2n)z^{-n}$, $X_1(z) = \sum x(2n+1)z^{-n}$, complex multiply: $2 \times (N/2)^2 + N$, complex add: $2 \times (N/2) \times (N/2-1) + N$,
 - Total: Multiples $\approx N/2 \log_2 N$, Adds $\approx N \log_2 N$

Complex Multiplication Optimization

- $(a+jb)(A+jB) = (aA-bB) + j(aB+bA) = \{cA-b(A+B)\} + j\{b(A+B)+dB\}$
- 3 real multiplies, 3 real adds.

Data Flow Graph (DFG)

- Nodes: computations e.g. add, multiply. (computation time)
- Directed Edges: data paths (no of delays (D))
- Critical Path: longest computation time among all zero-delay paths. Determines lower bound on the clock cycle ($T_s \geq \text{Critical Path}$).
- Loop Bound: loop computation time / total delays in loop.
- Iteration Bound (T_{∞}): maximum loop bound over all loops in the graph. Processing Rate $\leq 1/T_{\infty}$.
- Critical Loop: The loop with the maximum loop bound.

3.2 Pipelining

introduction:

- Goal: reduce critical path to increase clock frequency/sampling speed and maintains throughput by inserting latches (registers).
- Latency: no of clock cycles from input to corresponding output.

$$x(n) \xrightarrow{\text{latency}=m+1} y(n-m)$$

Pipelining option

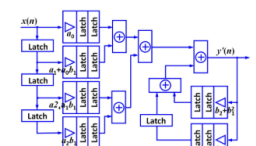
- 1: insert latches at a feed-forward cutset
- 2: fine-grain pipelining (multiplier: 10 gate delays $\rightarrow 6 + 4$ by inserting latches.
- 3: retiming (Transposed Form FIR Filter: Critical Path = $T_M + T_A$)

3.3 Retiming

- Goal: reduce critical path without increasing latency or adding extra registers by moving (D) around the graph. increase the throughput
- Cutset/Node Retiming: Add $k(> 0)$ delays to every edge $G_1 \rightarrow G_2$. Remove k delays from every edge $G_2 \rightarrow G_1$.
- Lattice Filter: critical path = $(N+1)T_A + 2T_M \rightarrow \text{retiming} \rightarrow 4T_A + 4T_M$.

3.4 Pipelined Recursive Filters

Higher Order Filters



- $y(n) = a_0 x(n) + a_1 x(n-1) + a_2 x(n-2) + b_1 y(n-1) + b_2 y(n-2)$, $y'(n) = y(n-2)$

Implementation

- Analyze the number of latches in the feedback loop m ($m = 3$)
- Rewrite the filter output without using $y(n-k)$ for $k < m$ (no $y(n-1)$ and $y(n-2)$)
- (Factorization in the feedforward loop: $(1+z^{-1}+z^{-2}+z^{-3})X(z) \rightarrow (1+z^{-2})\{(1+z^{-1})X(z)\}$)
- Draw the implementation structure with $y'(n)$
- Calculate p , the number of latches from $x(n)$ to $y'(n)$
- $y'(n) = y(n-p)$, latency = $p+1$

3.5 Parallel Processing (Unfolding)

Introduction

- Critical path unchanges. Hardware $\times N$. $T_s = T_c/N$, $T_c \geq \text{critical path}$. Throughput = L inputs/clock
- OR use slower, smaller hardware for same throughput.

Polyphase Parallel FIR Filter (L-phase):

$$\begin{bmatrix} Y_0 \\ Y_1 \\ Y_2 \end{bmatrix} = \begin{bmatrix} H_0 & z^{-1}H_2 & z^{-1}H_1 \\ H_1 & H_0 & z^{-1}H_2 \\ H_2 & H_1 & H_0 \end{bmatrix} \begin{bmatrix} X_0 \\ X_1 \\ X_2 \end{bmatrix}$$

- Hardware: $L \times N$ multipliers, $L \times (N-1)$ adders.

Low-Complexity Structure (2-phase example):

$$\begin{bmatrix} 1 & 0 & z^{-1} \\ -1 & 1 & -1 \end{bmatrix} \begin{bmatrix} H_0 & 0 & 0 \\ 0 & H_0 + H_1 & 0 \\ 0 & 0 & H_1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$Y_0(x(2n)) \xrightarrow{H_0} Y_1(x(2n)) \xrightarrow{H_1} Y_2(x(2n))$$

- Hardware: $3 \times N/2$ multipliers, $3 \times (N/2-1) + 4$ adders.

3.6 Real-Time Processing on a DSP Processor

Sample-by-Sample Processing

- T_s /Instruction Cycle Time = # of Instructions per T_s
- the digital processing of data $\leq T_s$
- Requires FIFO, memory, or register to hold incoming data.

Block Processing

- Collect N samples at a time.
- $T_{\text{processing}} < N \cdot T_s$, maximum block delay = $2N \cdot T_s$

Double buffering

- Advantage: process more data in T_s due to reduced setup time or higher F_s
- Disadvantage: more memory programming and processing latency

DSP Performance Ratings

- MIPS = number of Million Multiply-Accumulate (MAC) operations per second.
- Other metrics: BIPS (Billion Instructions Per Second), MOPS/BOPS(million operations per second), MFLOPS (Million Floating-Point Operations Per Second).
- $n_x(3 + n_h/2) + 25$, where n_x no of input samples in a

block, n_h no of filter coefficients, $/2$ processor has 2 MAC, 3 fetch/pipelined instrns, 25 overhead

Fixed-Point vs. Floating-Point Processors

- difference:
 - Precision, dynamic range, development time vs product cost
 - Word Width: 16/24-bit (fixed) vs. 32-bit (float).
 - Dynamic Range: fixed-point is limited; floating-point is large.
 - Programming: fixed-point requires scaling and overflow control; floating-point does not.
 - Compiler Efficiency: fixed-point often needs assembly; floating-point works well in C.
 - Development Time: fixed-point is longer; floating-point is shorter.
 - Clock Rate: fixed-point is typically faster.
 - Silicon Area & Cost: fixed-point is smaller and cheaper; floating-point is larger and costlier.
 - Power: fixed-point uses less power; floating-point uses more.
- Typical Applications
 - Fixed-Point: MP3 players, cell phones, digital cameras, motor control, disk drives, speech coding, modems.
 - Floating-Point: radar/sonar image processing, professional audio encoding/decoding/mixing, video coding/decoding, high-end sound synthesis, prototyping.

4. Unfolding and Folding (Hardware)

4.1 Unfolding

Algorithm for Unfolding:

1. For each U in original DFG, create J nodes: $U_0, U_1, \dots, U_{J-1}$.
2. For $U \rightarrow V$ with m delays in original graph, create J edges $U_i \rightarrow V_{(i+m)\%J}$ with $\left\lfloor \frac{i+m}{J} \right\rfloor$ delays, $i = 0, 1, \dots, J-1$.

Properties:

- each delay after unfolding equals J delays in the original
- $\left\lfloor \frac{0+m}{J} \right\rfloor + \left\lfloor \frac{1+m}{J} \right\rfloor + \dots + \left\lfloor \frac{J-1+m}{J} \right\rfloor = m$
- no of disjoint loops = $\gcd(m, J)$. Each disjoint loops: $m/\gcd(m, J)$ delays + $J/\gcd(m, J)$ copies of each node that appears in original loop.
- iteration bound $T_\infty \rightarrow J \cdot T_{\infty}$, $JT_s \geq$ critical path after unfolding

Applications:

- Case 1: longest node time $\geq T_\infty$. Case 2: T_∞ is not an integer. Case 3: both Case 1 & 2.

4.2 Parallel recursive filters

First Order Filters $y(n) = x(n) + ay(n-1)$

- Option 1 (block processing): Convert it to a parallel system with unfolding. Convert it to a parallel system with unfolding factor 2 (T_s unchanged)
- Option 2: Obtain disjoint loops, change D to 2D ($J=2$). Rewrite the filter output without using $y(n-1)$ but using $y(n-2)$ instead.
 - J multiply, J add, J sets. total J^2 multiply, J^2 add
- Option 3: Incremental block processing
 - Some outputs are computed using block processing (option 1), such as y_0 , other outputs are computed without using y_i , such as y_1 computed using delayed $y_1 = y(n-2)$ rather than y_0

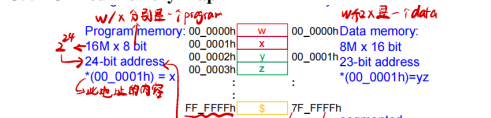
Higher Order Filters $y(n) = 2x(n) + 2y(n-1) - 2y(n-2)$

- IIR filter (N/M input/output order): inputs N multiply, N-1 add; past outputs M multiply, M add
- Option 1: Convert it to a parallel system with $J=3$
- Option 2: To obtain disjoint loops, change D to 3D ($J=3$).
 - inputs JM-M+N multiply, JM-M+N-1 add; past outputs M multiply, M add; J sets
- Option 3: Incremental block processing
 - y_2 computed without using y_1 (uses delayed y_2 instead), $y_2(n) = 2x(n) + 4x(n-1) + 2y(n-2) - 4y(n-3)$

5. C55x Processor (Software)

5.1 TMS320C55x Programming

C55x Unified Memory Map



- Unified memory map: program and data share same address space.
- $1K = 2^{10}$, $1M = 2^{20}$, $1G = 2^{30}$
- page address: 7 msb bits (128 pages, 00h-7Fh); address within page: 16 lsb bits (64K words, 0000h-FFFFh)

Registers

- Accumulators: AC0 to AC3 (40 bits)
 - guard=bits 39-32 | high=bits 31-26 | low=bits 15-0
 - AC0 40 bits, HI(AC0) 16 bits (31-16), LO(AC0) 16 bits (15-0)
- Auxiliary registers: AR0 to AR7
 - ARxH=bits 22-16 | ARx=bits 15-0; XARx=bits 22-0
 - AR0 16 bits (15-0), XAR0 23 bits
 - high_byte(*AR0): bits 15-8 of location at XAR0
- Temporary registers: T0 to T3 (16 bits)
 - Tx=bits 15-0
 - T0 16 bits
- Pair of registers: AR0, AR2, AR4, AR6, T0, T2
 - Refers to 2 registers such as AR0 and AR1
 - pair(T0): bits 31-16 to T0 and bits 15-0 to T1

Value

- k4/k4: 4-bit unsigned constant (#Bh = 11, #-11 = -11)
- K8: 8-bit signed constant (#20, #14h = 20)
- K16: 16-bit signed constant (#FC18h = -1000)

MOV instruction and data movement

- x bits $\rightarrow y$ bits, $x = y$: direct move.
- $x > y$: move y least significant bits (LSBs).
 - move bits 22-0 of AC0 should discard the 23th bit (9D=1001B $\rightarrow$ 001B=1D)
- $x < y$: sign-extend x bits to y bits.
 - Exception1: 23-bit XARx $\rightarrow$ 40-bit ACx: zero-fill.
 - Exception2: mov k4/k4 with 0/1 extension since known to be unsigned positive/negative
- no register to register MOV of 8 bits

Memory Addressing Modes in C55x

- Absolute (constant) addressing
 - k23 absolute addressing: *(#010501h) 16-bit data at 01_0501
 - k16 absolute addressing: *abs16(#0501h) 16-bit data at (DPH)_0501
- Direct (offset) addressing
 - XDP = 03_FFF0h, @65524=FFF4h $\rightarrow$ Doffset = 65524 - FFF0h = 4h, 16-bit data at address = DPH.(DP+Doffset) = 03_FFF4
 - If DP value is not known, use .dp x (MOV @(x+4), AR0) same as MOV @FFF4h, AR0).
 - * @(x+128) $\rightarrow$ XDP ???
- Indirect (pointer) addressing

- Auxiliary register (AR) indirect addressing:
 - * *AR0: 16-bit data at XAR0
 - * low_byte(*AR0): 8-bit (bit 7-0) data at XAR0, high_byte(*AR0): 8-bit (bit 15-8) data at XAR0
 - * *AR1: 2x16-bit data, 15-0 at XAR0, 31-16 at XAR1
 - * dbl(*AR0): 32-bit data. Bit 31-16 at XAR0, bit 15-0 at XAR0+1/XAR0-1 if XAR0 = even/odd.

Operands

- *ARx: no modification
- *ARx $\pm$: increase/decrease ARx after addressing
- * $\pm$ ARx: increase/decrease ARx before addressing
- *(ARx $\pm$ AR0): add/subtract AR0 to ARx after addressing
- *ARx(offset): no modification to ARx, use offset while addressing. offset = AR0/Ty/#K16. *AR1(#-1): AR1 - 1 before addressing
- *+ARx(#K16): add #K16(signed) to ARx before addressing

MAC m1, m2, sum

- sum = sum + (m1*m2). sum = ACx
- Process: operands sign-extended to 17 bits $\rightarrow$ 32-bit product $\rightarrow$ sign-extended to 40 bits $\rightarrow$ added to accumulator.
- Overflow: detected and sets accumulator overflow status bit.
- Types:
 1. MAC: with registers (m1 = AC0, only AC0₃₂₋₁₆ multiplied)
 2. MACK: with value
 3. MACM: with memory
 4. MACMK: with memory and value

Parallel Instructions in C55x

- MAC::MAC; MOV || MOV;
- Add/subtract, add/subtract; Add/subtract, read/write data memory; Multiply, multiply; Multiply, read/write data memory; MAC, MAC; MAC, multiply; MAC, read/write data memory

Pipeline in C55x

- Pipeline fetch phases (4): Prefetch1, Prefetch2, Fetch, Predecode.
- Execute phases (8): Decode, Address, Access1, Access2, Read, Execute, Write, Write+.
- To improve cycle utilization, different phases of different instructions can be simultaneously executed.

Circular Buffer Addressing

- Pointer is incremented modulo the buffer size
- Requires buffer start address, buffer size, and pointer (ARx or CDP).

5.2 TMS320C55x architecture

Overflow Detection (Q0.15 Format)

- No overflow:
 - 5000_h + 3000_h = 8000_h (0.625 + 0.375 = -1) $\rightarrow$ sum of two positives yields negative $\Rightarrow$ overflow.
- C000_h + B000_h = 7000_h (-0.5 + (-0.625) = 0.875) $\rightarrow$ sum of two negatives yields positive $\Rightarrow$ overflow.
- Overflow gives wrap around effect for 2's complement numbers: A large positive number becomes a large negative number, vice versa

Saturation

- D-unit (AC): positive $\rightarrow$ 00_7FFF_FFFF_h, negative $\rightarrow$ FF_8000_0000_h.
- A-unit: positive $\rightarrow$ 7FFF_h, negative $\rightarrow$ 8000_h.
- If saturation on multiplication bit (SMUL) is 1, then same as SATD for multiplication result
- Saturation clips the error: A large positive number becomes a slightly smaller positive number, and vice versa, introducing less error

Fractional Mode Multiplication

- Process:
 1. Sign extend 16-bit Q0.15 inputs to 17-bit Q1.15 format
 2. Multiply to obtain 32-bit Q1.30 product.
 3. shift left by 1 bit to obtain 32-bit Q0.31 fraction
 - integer mode: no shift, only sign_extend(P)
 4. If FRCT=1, result is automatically left-shifted by 1 bit $\rightarrow$ Q0.31 format.

6. Finite Precision Implementation, ADC and DAC

6.1 Fixed point and floating point number representations

Two's Complement

- Sign extension: (1011)₂s = (11101)₂s = (1...101)₂s
- range: -2^{N-1} to $2^{N-1} - 1$
- Multiplication of 2 n-bit numbers gives a 2n bit product.
 - use sign extension to 2n bits for negative
 - Discard bits beyond the 2n-th

Fixed point number:

- Qm.n format: n fractional bits, N = n+m+1 bit number
- Dynamic Range (DR):

$$DR_{dB} = 20 \log_{10} \left(\frac{\text{Largest number}}{\text{Smallest Positive number excluding 0}} \right)$$

- Any 16-bit 2's complement Qm.n has $DR \approx 90$ dB.
- Precision = difference between 2 consecutive numbers
- Adding two N-bit Qm.n numbers $\rightarrow$ (N+1)-bit Q(m+1).n
- Multiplying two N-bit Qm.n numbers $\rightarrow$ 2N-bit Q(2m+1).(2n)

Floating point number

- $A = P \times Q^D$, P fraction, mantissa; Q base, radix; D exponent, characteristic
- mantissa | exponent
- the first bit in mantissa is always "1", so it is ignored

Floating Point (IEEE 754):

- Structure: S (1 bit) - Exp (8 bits) - D (23 bits).
- Value: $(-1)^S \cdot 2^{(\text{Exp}-2^E-1)} \cdot (1.D)$, $2^{E-1} = 127$
 - 23-bit mantissa: 01100... D = 0.375 mantissa value M=1.375
 - Denormalized number = $(-1)^S \times 2^{-(Bias-1)} \times D$
- Smallest: 1.176×10^{-38} , Largest: 3.403×10^{38} .
- Dynamic Range: 1529 dB (254 $\times$ 6.02 dB).
- Precision: 2^{-23} (same exponent).
- Fixed vs. Floating:
 - Floating: larger dynamic range (exponent).
 - Fixed: finer precision (more mantissa bits).

6.2 Quantization, Truncation, and Round-off

Quantizer:

- Mid-tread: Levels = $2^n - 1$, $Q = (V_{\max} - V_{\min}) / (2^n - 1)$.
- Mid-rise: Levels = 2^n , $Q = (V_{\max} - V_{\min}) / 2^n$.

Rounding:

- Error $e(n)$: uniform over $[-Q/2, Q/2]$, pdf=1/Q.
- Mean: $\eta_e = 0$. Variance: $\sigma_e^2 = Q^2/12$.
- RMS Error: $\approx 0.29Q$.

Truncation:

- Error $e(n)$: uniform over $[-Q, 0]$, pdf=1/Q.
- Mean: $\eta_e = -Q/2$. Variance: $\sigma_e^2 = Q^2/12$.

Additive Noise Model:

- $y_{\text{quant}}[n] = x[n] + e[n]$
- $e(n)$ is white noise (uncorrelated with $x(n)$ and itself).

$$S_e(e^{j\omega}) = \sigma_e^2 = Q^2/12.$$

SQNR:

$$SQNR_{dB} = 10 \log_{10}(\sigma_x^2 / \sigma_e^2) = 6.02n + 4.77 - 20 \log_{10}(V_{\max} / \sigma_x)$$

- For speech/music: $\sigma_x = V_{\max} / 4$, $SQNR_{dB} = 6.02n - 7.27$.
- Each additional bit: +6 dB SQNR.
- Oversampling: doubling rate $\Rightarrow$ +3 dB SQNR.

Dithering:

- Purpose: make quantization error white for constant/slow signals.
- add small random noise (uniform/triangular/Gaussian) before ADC.
- Advantage: provide more information about the original signal.
- Disadvantage: increases total output noise.
- Subtractive Dithering:
 - Generate noise $\rightarrow$ DAC $\rightarrow$ Add $\rightarrow$ ADC $\rightarrow$ Subtract.
 - Advantage: does NOT increase final output noise.

Error Analysis

- Input noise $e(n) \xrightarrow{H(e^{j\omega})}$ Output noise $v(n)$
- Mean: $\eta_v = H(e^{j0}) \cdot \eta_e$ (DC gain $\times$ input mean).
- Variance: $\sigma_v^2 = \frac{1}{2\pi} \int_{-\pi}^{\pi} |H(e^{j\omega})|^2 S_e(e^{j\omega}) d\omega$.
- PSD: $S_{out}(e^{j\omega}) = |H(e^{j\omega})|^2 S_{in}(e^{j\omega})$.
- If $e(n)$ white and H has passband 2Δ : $\sigma_v^2 = \frac{\Delta}{\pi} \sigma_e^2$.
- Uncorrelated Noise Sum:
 - Mean: $\eta_{\text{total}} = \sum \eta_i$, Variance: $\sigma_{\text{total}}^2 = \sum \sigma_i^2$.
 - Sum of white noises $\rightarrow$ white noise.

FIR Filter Quantization Noise:

- Sources (uncorrelated):
 - Input: $e_x(n)$, $\sigma_x^2 = \frac{Q_x^2}{12}$
 - Multiply-Add: $e_k(n)$ ($k = 0, \dots, N-1$), each $\sigma_m^2 = \frac{Q^2}{12}$
 - Output: $e_y(n)$, $\sigma_y^2 = \frac{Q_y^2}{12}$
- total output mean: $\eta_{\text{total}} = H(e^{j0})\eta_x + \eta_0 + \dots + \eta_{N-1} + \eta_y$
- Variance: $\sigma_{\text{total}}^2 = \sigma_x^2 \cdot \|H(e^{j\omega})\|_2^2 + \sum_{k=0}^{N-1} \sigma_m^2 + \sigma_y^2$
- power spectral density: $S_{\text{total}}(e^{j\omega}) = |H(e^{j\omega})|^2 S_x(e^{j\omega}) + S_0(e^{j\omega}) + \dots + S_{N-1}(e^{j\omega}) + S_y(e^{j\omega})$
- If all rounding with step Q : $\sigma_{\text{total}}^2 = \frac{Q^2}{12} (\frac{\Delta}{\pi} + N + 1)$

6.3 Analog to digital converters

Counter ADC

- count Q by Q
- Advantages: simple design, low cost.
- Disadvantages: slow conversion speed, depends on magnitude of V_{IN} .

Successive Approximation Register (SAR) ADC

- count from MSB to LSB
- Advantages: fast (exactly n clock cycles for n -bit resolution), high accuracy, moderate cost.
- Disadvantages: performance relies on linearity and accuracy of the internal DAC.

Dual-Slope Integrating ADC

- $V_{in} \times T_{fixed} = V_{ref} \times T_{counter}$
- Advantages: very high precision and linearity; highly immune to noise, temperature drift, and component aging.
- Disadvantages: very slow due to long integration times.

Parallel (Flash) ADC

- $V_{max} = 1 - Q$, $Q = \frac{V_{max} - V_{min}}{2^n}$, $V_{ref} = 1 - \frac{Q}{2}$, $ground = -1 + \frac{Q}{2}$, $P - \frac{Q}{2} < V_{in} < P + \frac{Q}{2}$, $output = P$
- Advantages: extremely fast conversion speed; limited only by propagation delays.
- Disadvantages: high power consumption, large silicon area, and cost that grows exponentially with resolution ($2^N - 1$ comparators). Limited to low resolutions (typically ≤ 8 bits).

Sigma-Delta ($\Sigma\Delta$) ADC

- Principle and Performance
 - Core Idea: uses 1-bit quantizer with oversampling and noise shaping to push quantization noise to high frequencies.
 - SQNR improvement per doubling of f_s :
 - * adding 1 bit resolution = +6dB SQNR.
 - * Oversampling only: +3 dB (equivalent to +0.5 bits).
 - * First-order noise shaping: +9 dB (equivalent to +1.5 bits).
 - * Second-order noise shaping: +15 dB (= +2.5 bits).
 - Internal Sampling Rate: = $f_s \times 2^N$, where N is number of doublings needed to achieve target resolution (in bits from 1-bit quantizer (N=15/1.5 or 15/2.5)).
- Advantages of $\Sigma\Delta$ ADC:
 - Low cost (90% digital circuitry).
 - No need for high-precision analog components.
 - High resolution and excellent noise characteristics.
 - Anti-aliasing filter requirement relaxed (single-stage RC sufficient).
- Disadvantages of $\Sigma\Delta$ ADC:
 - Low speed due to high oversampling (used for audio/voice).
 - Slowly varying inputs can cause low-frequency artifacts.

6.4 Digital to analog converters

R-2R Ladder DAC

- Input: N -bit binary number $a_N \dots a_1$.
 - Output voltage: $V_o = V_{ref} \left(\frac{a_1}{2} + \frac{a_2}{4} + \dots + \frac{a_N}{2^N} \right)$.
 - Range: $0 \leq V_o < V_{ref}$.
 - Requires high-precision resistor matching.
- Zero-Order Hold (ZOH)
 - Holds each digital sample value constant until the next sample.
 - Produces a staircase waveform.
- Sigma-Delta DAC
 - Input upsampled, then quantized by 1-bit quantizer.
 - Quantization error is fed back in a loop (noise shaping).
 - Advantages: low-cost digital processing; simple analog reconstruction filter.